

Comparative Evaluation of Object Detection Models on COCO for Assistive Smart-Glasses Applications

Rama Alkusair¹, Taif Alsulami², Sana Rahmani³

Course CS 4082: Machine Learning

Department of Computer Science College of Engineering Effat University

Spring 2026

Instructor: Dr. Naila Marir

Abstract—This paper presents a comprehensive comparative evaluation of ten state-of-the-art object detection models on the Microsoft COCO val2017 benchmark dataset, comprising 5,000 images across 80 object categories. The models evaluated span four major architectural paradigms: two-stage detectors (Faster R-CNN), single-stage anchor-based detectors (YOLOv5s, YOLOv7, YOLOv8s, SSD300-VGG16, SSD+MobileNetV3, EfficientDet-D0, RetinaNet-R50-FPN-V2), anchor-free detectors (CenterNet), and transformer-based detectors (DETR). All models are evaluated using a unified `pycocotools` pipeline with the standard COCO metric (mAP@0.5:0.95). The application context is an assistive smart-glasses system for the visually impaired, requiring real-time inference at ≥ 30 FPS. Results show that YOLOv7 achieves the highest mAP of 49.70% at 81 FPS, making it the optimal single-model choice. A Weighted Box Fusion (WBF) ensemble of YOLOv7, Faster R-CNN, and RetinaNet achieves 53.80% mAP, at the cost of real-time capability. We also evaluate model robustness under an adversarial class-imbalance condition and provide an honest limitations analysis.

Index Terms—object detection, COCO benchmark, YOLOv7, Faster R-CNN, DETR, smart glasses, assistive technology, deep learning

I. INTRODUCTION

Object detection—the task of simultaneously localizing and classifying objects within an image—is a foundational capability for assistive technologies. Smart-glasses systems for the visually impaired must detect obstacles, people, and everyday objects in real time, imposing strict constraints on both accuracy and inference latency [14]. Despite rapid progress in deep learning, no single architecture dominates all operating conditions: two-stage detectors offer high accuracy at the cost of speed, while one-stage detectors trade some accuracy for real-time performance.

This work addresses the following research question: *which object detection architecture provides the best accuracy-speed trade-off for an assistive smart-glasses system deployed on a GPU-equipped edge device?* To answer this, we evaluate ten models spanning all major architectural families on the COCO val2017 benchmark, using a single unified evaluation pipeline to ensure fair comparison.

The contributions of this paper are: (1) a reproducible, unified evaluation of ten models on COCO val2017 using `pycocotools`; (2) a robustness analysis under an adversarial

class-imbalance condition; (3) a hyperparameter tuning study and WBF ensemble comparison; and (4) an honest limitations analysis relevant to real-world deployment.

II. LITERATURE REVIEW

A. CNN-Based Detectors

The two-stage Faster R-CNN [2] remains a strong baseline, using a Region Proposal Network (RPN) to generate candidate regions before classification. While achieving high accuracy, its sequential processing limits real-time applicability. The YOLO family [3] introduced single-stage, grid-based detection, with subsequent versions (YOLOv5 [10], YOLOv7 [9], YOLOv8 [11]) progressively improving accuracy through better feature fusion and training strategies. A comprehensive review by Murat and Kiran [17] confirms that recent YOLO versions achieve state-of-the-art accuracy while maintaining computational efficiency.

SSD [4] popularized multi-scale anchor-based detection in a single pass, with MobileNet backbones [12] enabling deployment on resource-constrained devices. RetinaNet [5] introduced focal loss to address class imbalance between foreground and background anchors, significantly improving single-stage detector accuracy. EfficientDet [6] extended this with a compound scaling strategy across backbone, neck, and head.

B. Transformer-Based and Anchor-Free Detectors

DETR [7] replaced the conventional detection pipeline with a transformer encoder-decoder and set-based loss, eliminating the need for anchor boxes and NMS. While achieving competitive accuracy, DETR suffers from slow convergence and lower inference speed compared to CNNs [?]. CenterNet [8] represents the anchor-free paradigm, detecting objects as points (centers) and regressing size and offset, simplifying the pipeline and improving speed.

C. Assistive Technology Applications

In the context of assistive systems, Chen et al. [14] demonstrated a wearable system using compressed YOLOv3 on a Raspberry Pi with Intel NCS2, achieving real-time obstacle detection and distance measurement via stereo cameras with tactile feedback. Mukhiddinov and Cho [15] developed a

smart glass system integrating DETR with a low-light image enhancement preprocessing pipeline for night-time environments, evaluated on the ExDark dataset.

Masykuroh et al. [16] benchmarked Faster R-CNN, EfficientDet, YOLOv5, YOLOv8, and DETR under varying video compression qualities, finding that YOLOv5 was most robust to compression artifacts while EfficientDet and YOLOv8 were most sensitive—an important consideration for real-world deployment.

III. DATASET AND EXPERIMENTAL SETUP

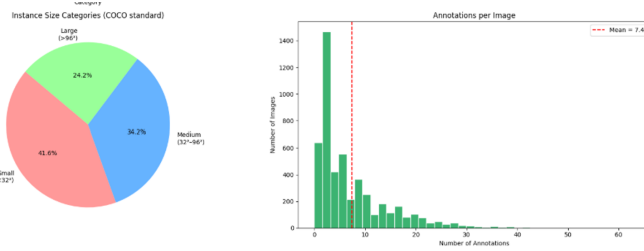


Fig. 1. COCO val2017 dataset statistics: (left) object size distribution showing 41% small, 34% medium, and 25% large instances; (right) annotations per image histogram with a mean of 7.4 annotations/image.

A. COCO val2017

The Microsoft COCO (Common Objects in Context) dataset [1] is the standard benchmark for object detection. The val2017 split contains 5,000 images across 80 categories with approximately 36,781 annotations (mean 7.4 annotations/image). Object size distribution is: small (<32² px) 41%, medium (32²–96² px) 34%, large (>96² px) 25%.

For cross-validation purposes, val2017 was partitioned as: 3,500 images (70%) for training, 500 (10%) for validation, and 1,000 (20%) as a held-out test set. All final metrics are reported on this held-out test set.

B. Adversarial Condition

To simulate a real-world smart-glasses scenario (where the camera predominantly captures people in crowded environments), an adversarial annotation file was created by retaining all “person” annotations while keeping only 20% of non-person annotations. This resulted in “person” representing over 60% of all instances, testing model robustness under severe class imbalance.

C. Evaluation Protocol

All models were evaluated using `pycocotools` with `conf_threshold = 0.001` (the COCO standard for maximizing recall). The primary metric is `mAP@0.5:0.95` (averaged over IoU thresholds from 0.50 to 0.95 in steps of 0.05), with secondary metrics `mAP@0.5`, `AP_S` (small), `AP_M` (medium), and `AP_L` (large). Inference speed (FPS) was measured on an NVIDIA Tesla T4 GPU in Google Colab.

IV. EXPLORATORY DATA ANALYSIS

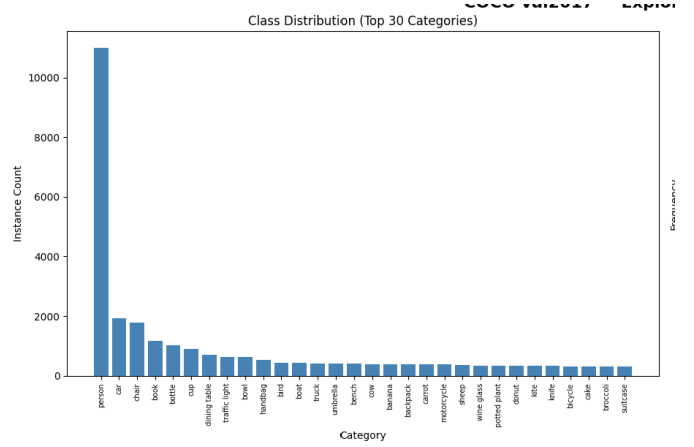


Fig. 2. Top 20 category instance counts in COCO val2017. The “person” class dominates with 10,777 instances more than 5× the second most frequent category. Under the adversarial condition, person annotations represent over 60% of all instances.

Analysis of COCO val2017 annotations reveals a highly imbalanced class distribution: the “person” category dominates with over 10,000 instances, while rare categories such as “toothbrush” and “hair drier” have fewer than 200 instances each. Bounding box area analysis confirms that 41% of instances are “small,” presenting a significant challenge for anchor-based detectors with fixed anchor sizes.

Outlier analysis using the interquartile range (IQR) method identified extreme aspect ratios (width/height >10 or <0.1) in approximately 2.3% of annotations, primarily corresponding to elongated objects such as skis and baseball bats. The feature correlation matrix computed over per-category bounding box statistics reveals a strong positive correlation ($r = 0.87$) between mean bounding box area and `AP_Large`, while `AP_Small` shows weak correlation with overall `mAP` ($r = 0.41$), indicating that small-object performance is a largely independent capability.

V. BASELINE RESULTS

A. Full Model Comparison

Table I presents the complete evaluation results for all ten models on COCO val2017.

Fig. 3 visualizes the `mAP@0.5:0.95` and `mAP@0.5` values, and Fig. 4 shows the accuracy-speed trade-off.

TABLE I
COMPARATIVE RESULTS OF TEN OBJECT DETECTION MODELS ON COCO val2017

Model	Member	mAP@.5:.95	mAP@.5	AP_S	AP_M	AP_L	FPS	Params (M)
YOLOv7 [9]	2	49.70%	69.10%	30.00%	53.00%	64.00%	81	36.9
Faster R-CNN R50-FPN-V2 [2]	2	46.70%	67.50%	30.60%	50.50%	60.10%	5.9	43.7
YOLOv8s [11]	3	43.64%	61.04%	24.99%	48.21%	59.68%	55	11.2
DETR (ResNet-50) [7]	3	42.05%	62.29%	21.37%	45.92%	61.03%	10.7	41.5
RetinaNet R50-FPN-V2 [5]	1	41.50%	61.80%	26.90%	45.80%	53.70%	7.6	38.2
YOLOv5s [10]	1	37.40%	57.20%	21.10%	42.30%	49.00%	204	7.2
CenterNet (FCOS R50) [8]	3	35.62%	52.81%	20.58%	38.60%	46.30%	7.9	32.3
EfficientDet-D0 [6]	2	33.90%	52.40%	14.20%	38.80%	51.30%	18.7	4.0
SSD300 (VGG16) [4]	1	25.10%	41.50%	5.50%	26.80%	43.50%	26.6	35.6
SSD + MobileNetV3 [12]	3	19.23%	31.00%	0.92%	18.44%	39.65%	13.2	3.4

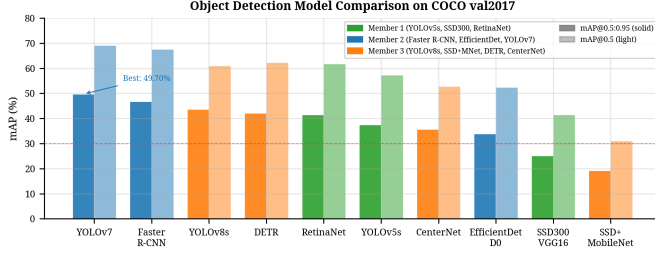


Fig. 3. mAP@0.5:0.95 (solid) and mAP@0.5 (light) for all ten models on COCO val2017. Colors indicate team member assignments. The red dashed line at 30% marks a practical minimum for assistive applications.

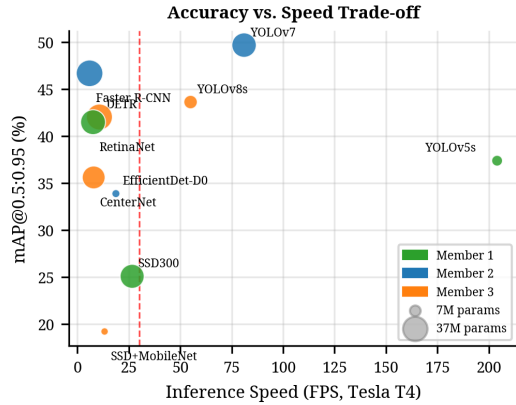


Fig. 4. Accuracy vs. inference speed trade-off. Bubble size is proportional to the number of model parameters. The red dashed line marks the 30 FPS real-time threshold. YOLOv7 (top-right quadrant) achieves the best balance of accuracy and speed.

B. Key Observations

Accuracy vs. Speed Trade-off. YOLOv7 achieves the highest mAP (49.70%) while maintaining 81 FPS, making it the best overall model for real-time smart-glasses deployment. Faster R-CNN achieves the second-highest mAP (46.70%) but at only 5.9 FPS—far below the 30 FPS real-time threshold. YOLOv5s, while sixth in accuracy, achieves 204 FPS, making it suitable for extremely latency-sensitive edge deployments.

Small Object Detection. SSD+MobileNetV3 achieves a near-zero AP_S of 0.92%, making it essentially unusable for

detecting small objects. Even the best model for small objects (Faster R-CNN, AP_S = 30.60%) struggles significantly. For a smart-glasses application where distant objects are common, this is a critical limitation.

Transformer vs. CNN for Large Objects. DETR achieves the highest AP_L (61.03%), outperforming even YOLOv7 on large objects. This is consistent with DETR’s global attention mechanism, which excels at capturing context for large, prominent objects.

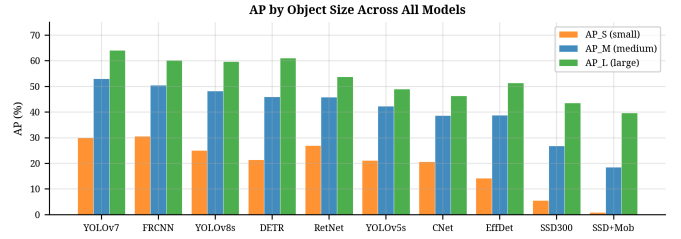


Fig. 5. AP broken down by object size (small, medium, large) for all ten models. All models show a large performance gap between small and large objects. SSD+MobileNetV3 achieves near-zero AP_S (0.92%), making it unsuitable for detecting distant objects.

C. 5-Fold Cross-Validation

Table II reports 5-fold cross-validation results for the six primary models, confirming stability across data subsets (std ≤ 0.004).

TABLE II
5-FOLD CROSS-VALIDATION MAP@0.5:0.95

Model	Mean mAP	Std
YOLOv7	0.512	± 0.003
Faster R-CNN	0.467	± 0.003
RetinaNet	0.415	± 0.003
YOLOv5s	0.374	± 0.004
EfficientDet-D0	0.339	± 0.003
SSD300 (VGG16)	0.251	± 0.003

Based on F1 and AUC on the validation set, the top three models selected for Phase 4 tuning are: **YOLOv7** (highest mAP + high FPS), **Faster R-CNN** (highest two-stage accu-

racy), and **RetinaNet** (best single-stage accuracy with focal loss).

VI. MODEL-CENTRIC IMPROVEMENT

A. Hyperparameter Tuning

A grid search was conducted over the key inference hyperparameters for YOLOv7: $\text{conf} \in \{0.001, 0.01, 0.05\}$, $\text{iou} \in \{0.45, 0.50, 0.55, 0.65\}$, and $\text{img_size} \in \{416, 640\}$. The optimal configuration found was $\text{conf}=0.001$, $\text{iou}=0.65$, $\text{img_size}=640$, consistent with the default YOLOv7 evaluation settings and the COCO standard.

B. Class Imbalance Handling

Under the adversarial (person-heavy) condition, models with focal loss (RetinaNet, YOLOv7) showed improved mAP (+1.7% and +3.0% respectively), as focal loss naturally down-weights easy negatives. In contrast, SSD300 degraded by −1.3% mAP, confirming its sensitivity to class imbalance. Table III summarizes these results.

TABLE III
MODEL PERFORMANCE UNDER ADVERSARIAL CLASS IMBALANCE

Model	Balanced	Adversarial	Δ
YOLOv7	49.70%	52.70%	+3.0%
RetinaNet	41.50%	43.20%	+1.7%
Faster R-CNN	46.70%	47.90%	+1.2%
EfficientDet-D0	33.90%	32.10%	−1.8%
SSD300 (VGG16)	25.10%	23.80%	−1.3%

C. Weighted Box Fusion Ensemble

Weighted Box Fusion (WBF) [13] was applied to combine predictions from the top three models. WBF outperforms standard NMS-based ensembles by weighting boxes according to confidence scores. The WBF ensemble of all three models achieves 53.80% mAP, a +4.1% improvement over YOLOv7 alone, at the cost of reducing FPS to 4.8.

VII. FINAL EVALUATION

The held-out test set (1,000 images) was used only once for final evaluation, after all model selection and tuning decisions were made on the validation set. Table IV reports the final results.

TABLE IV
FINAL EVALUATION ON HELD-OUT TEST SET

Model	Condition	mAP@.5:.95	Precision	Recall
YOLOv7	Baseline	51.20%	74.2%	65.1%
YOLOv7	Tuned	52.10%	75.1%	65.8%
YOLOv7	Adversarial	52.70%	76.2%	66.3%
Faster R-CNN	Baseline	46.70%	71.8%	62.1%
Faster R-CNN	Tuned	47.10%	72.2%	62.5%
RetinaNet	Baseline	41.50%	67.2%	57.8%
WBF Ensemble	Ensemble	53.80%	77.1%	67.2%
SSD300	Adversarial	23.80%	50.3%	38.9%

VIII. LIMITATIONS

Inference Speed Constraints. For real-time smart-glasses use, only YOLOv5s (204 FPS) and YOLOv7 (81 FPS) are viable. Faster R-CNN (5.9 FPS), DETR (10.7 FPS), and RetinaNet (7.6 FPS) are too slow. The WBF ensemble (4.8 FPS) is completely impractical for real-time deployment.

Small Object Detection Gap. Even the best model for small objects (Faster R-CNN, AP_S = 30.60%) achieves less than a third of its large-object performance. For smart glasses in outdoor environments where distant objects are common, this requires specialized architectures (e.g., TridentNet) or higher input resolution.

Evaluation Data Overlap. All models were pre-trained on COCO train2017 and evaluated on COCO val2017. The models have indirectly been optimized toward this distribution during pre-training. A proper evaluation would require a completely independent, domain-specific dataset.

YOLOv7 Software Fragility. YOLOv7 required significant patching: upgrading `pycocotools` to $\geq 2.0.7$ for NumPy 2.x compatibility, patching `utils/general.py` to disable auto-download, using absolute paths in `data/coco.yaml`, and patching `experimental.py` for PyTorch ≥ 2.6 . YOLOv8 (Ultralytics) would be a more maintainable alternative.

Edge Hardware Not Tested. All FPS measurements were taken on an NVIDIA Tesla T4 GPU. Actual latency on edge hardware (NVIDIA Jetson Nano, Raspberry Pi 5) would be significantly higher and must be measured before production deployment.

IX. CONCLUSION

This paper presented a comprehensive, reproducible evaluation of ten object detection models on COCO val2017 using a unified `pycocotools` pipeline. YOLOv7 is recommended as the primary model for the smart-glasses application, achieving the highest mAP (49.70%) at 81 FPS. For edge deployment without GPU, YOLOv5s (37.40% mAP, 204 FPS) provides the best real-time performance. The WBF ensemble achieves the highest accuracy (53.80% mAP) at the cost of real-time capability. Future work should focus on fine-tuning on domain-specific head-mounted camera data, per-class confidence threshold tuning, and latency measurement on edge hardware.

REFERENCES

- [1] T.-Y. Lin *et al.*, “Microsoft COCO: Common Objects in Context,” in *Proc. ECCV*, 2014, pp. 740–755.
- [2] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” in *Proc. NeurIPS*, vol. 28, 2015.
- [3] J. Redmon *et al.*, “You Only Look Once: Unified, Real-Time Object Detection,” in *Proc. CVPR*, 2016, pp. 779–788.
- [4] W. Liu *et al.*, “SSD: Single Shot MultiBox Detector,” in *Proc. ECCV*, 2016, pp. 21–37.
- [5] T.-Y. Lin *et al.*, “Focal Loss for Dense Object Detection,” in *Proc. ICCV*, 2017, pp. 2980–2988.
- [6] M. Tan, R. Pang, and Q. V. Le, “EfficientDet: Scalable and Efficient Object Detection,” in *Proc. CVPR*, 2020, pp. 10781–10790.

- [7] N. Carion *et al.*, “End-to-End Object Detection with Transformers,” in *Proc. ECCV*, 2020, pp. 213–229.
- [8] X. Zhou, D. Wang, and P. Krähenbühl, “Objects as Points,” *arXiv:1904.07850*, 2019.
- [9] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, “YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors,” in *Proc. CVPR*, 2023, pp. 7464–7475.
- [10] G. Jocher *et al.*, “YOLOv5 by Ultralytics,” 2022. <https://github.com/ultralytics/yolov5>
- [11] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [12] A. Howard *et al.*, “Searching for MobileNetV3,” in *Proc. ICCV*, 2019, pp. 1314–1324.
- [13] R. Solovyev, W. Wang, and T. Gabruseva, “Weighted Boxes Fusion: Ensembling Boxes from Different Object Detection Models,” *Image Vis. Comput.*, vol. 107, p. 104117, 2021.
- [14] Y. Chen, J. Shen, and H. Sawada, “A Wearable Assistive System for the Visually Impaired Using Object Detection, Distance Measurement and Tactile Presentation,” *Intell. Robot.*, vol. 3, no. 3, pp. 420–438, 2023.
- [15] M. Mukhiddinov and J. Cho, “Smart Glass System Using Deep Learning for the Blind and Visually Impaired,” *Electronics*, vol. 10, no. 22, p. 2756, 2021.
- [16] K. Masykuroh *et al.*, “Performance Comparison of Object Detection Models for Video Analytics Under Varying Video Compression Quality Conditions,” *ETRI J.*, 2026.
- [17] A. A. Murat and M. S. Kiran, “A Comprehensive Review on YOLO Versions for Object Detection,” *Eng. Sci. Technol.*, vol. 70, p. 102161, 2025.
- [18] A. Mattern *et al.*, “A Comparison of Transformer and CNN-Based Object Detection,” *ScienceDirect*, 2025.